

URL Shortener Service

Final Proje Raporu

Ders: MTH2526-B25 - Bulut Mimarilerinde Test Muhendisligi
Donem: 2025-2026 Bahar Yariyili
Egitmen: Busra Ayaksiz
Konu: #1 - URL Shortener Service
Tarih: 2 Haziran 2026

Grup Uyeleri

Isim	Ogrenci No	RoI
Talha Ergelen	-	Tech Lead / Repo Sahibi
Osman Cingoz	-	Backend & DevOps

GitHub: https://github.com/talhaergelen/bmtm-url_shortener_service

1. Giris

1.1 Projenin Amaci

Bu projenin amaci, ders boyunca edinilen test muhendisligi bilgi ve araclarini birlestirerek kucuk bir mikroservice endustri standardinda bir uctan uca (E2E) test boru hattı kurmaktır. Konu olarak URL Shortener Service (Kisa Link Servisi) secilmistir. Bu servis uzun URL'leri 6 karakterlik kısa kodlara donusturur, HTTP 301 ile yonlendirme yapar ve her tiklamayi kayit altina alarak istatistik sunar.

1.2 Neden Bu Konu?

URL kisaltma servisi, CRUD (Olustur/Oku/Guncelle/Sil) operasyonlarinin tamamini dogal olarak icermesi, yonlendirme mantiginin test edilebilirliğı ve istatistik toplama ozelligi sayesinde cok katmanli test stratejisini uygulamak icin ideal bir alan sunmaktadir. Servisin basitligi, odagi uygulama karmasikligindan test altyapisi kalitesine kaydirmamiza olanak tanimistir.

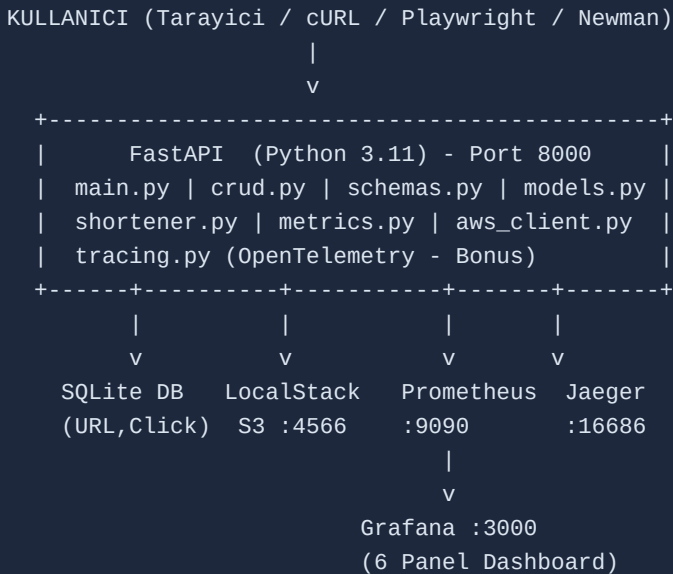
1.3 Teknoloji Yigini

- * Backend: Python 3.11, FastAPI, SQLAlchemy ORM, Pydantic v2
- * Veritabani: SQLite (gelistirme) + PostgreSQL (Testcontainers ile entegrasyon testi)
- * Test: Pytest, Factory Boy, Faker, Playwright, Newman/Postman, k6
- * Konteyner: Docker (Multi-stage), docker-compose (5 servis)
- * Orkestrasyon: Kubernetes (Minikube), Helm Chart (Bonus)
- * CI/CD: GitHub Actions (7 job'lik pipeline)
- * Izleme: Prometheus, Grafana (6 panel), OpenTelemetry + Jaeger (Bonus)
- * Bulut: LocalStack (AWS S3 emulasyonu)

2. Sistem Mimarisi

2.1 Mimari Diyagram

Asagidaki sema, sistemin tum bileşenlerini ve aralarındaki veri akisini gostermektedir. Diyagramin PNG versiyonu docs/architecture.png dosyasinda yer almaktadır.



2.2 Bilesen Aciklamalari

Bilesen	Teknoloji	Aciklama
API Sunucusu	FastAPI+Uvicorn	8 REST endpoint, asenkron ASGI
Veritabani	SQLAlchemy+SQLite	URL ve Click: 2 entity, ORM
Bulut Depolama	LocalStack S3	Istatistik JSON upload
Monitoring	Prometheus+Grafana	Metrik toplama, 6 panel
Tracing (Bonus)	OpenTelemetry+Jaeger	Dagitik izleme, span takibi
Konteyner	Docker Multi-stage	~150 MB imaj, non-root user
Orkestrasyon	K8s (Minikube)	2 replika, NodePort, ConfigMap

2.3 REST API Endpoint'leri

Method	Endpoint	Aciklama	Kod
GET	/health	Saglik kontrolu (liveness)	200
POST	/shorten	Yeni kısa URL olustur	201
GET	/short_code	Orijinal URL'ye yonlendir	301
GET	/urls/list	Tum URL'leri listele	200
GET	/urls/{code}	Tek URL detayi	200
GET	/stats/{code}	Tiklama istatistikleri	200
DELETE	/urls/{code}	URL silme	200
GET	/metrics	Prometheus metrikleri	200

3. Test Stratejisi

3.1 Test Piramidi Yorumu

Projede klasik Test Piramidi yaklasimi benimsenmistir. Piramidin tabaninda hizli ve izole calisan birim testleri, ortasinda API ve veritabani entegrasyon testleri, tepesinde ise tarayici tabanlı E2E testleri yer almaktadır.

```

/\      E2E (Playwright)      --> 5 senaryo
 /  \      Postman / Newman    --> 8 istek
/----\      Integration (API + DB) --> 39 test
 /    \      Unit (is mantigi)   --> 42 test
/-----\
 /  TOPLAM:  \      87 test + 8 Postman = 95 kontrol
/-----\      Coverage: >= %90

```

3.2 Birim Testler (Unit) - 42 test

Dosya	Test Sayisi	Kapsam
test_shortener.py	12	Kisa kod uretimi, benzersizlik, uzunluk
test_crud.py	19	CRUD islemleri, edge case'ler
test_aws.py	11	S3 istemci mock, baglanti hatasi senaryolari

Izolasyon: Her test SQLite in-memory veritabani ile calisir. conftest.py icinde yield bazli fixture ile test sonrasi temizlik yapilir. Veri Uretimi: Factory Boy ve Faker kutuphaneleri ile URLFactory sinifi olusturulmustur.

3.3 Entegrasyon Testler - 39 test

Dosya	Test Sayisi	Kapsam
test_api.py	31	8 endpoint'in TestClient ile sinanmasi
test_database.py	8	Testcontainers ile PostgreSQL CRUD

Testcontainers kutuphanesi ile Docker üzerinde gecici PostgreSQL konteyneri otomatik baslatilir, testler kosturulur ve konteyner imha edilir. Bu, gercek veritabani davranisini test ortaminda simule eder.

3.4 E2E Testler (Playwright) - 5 senaryo

#	Senaryo	Aciklama
1	Ana sayfa yukleme	Form elemanlari gorunur mu?
2	URL kisaltma	Sonuc kutusu aciliyor mu?
3	Gecersiz URL	Hata mesajı gösteriliyor mu?
4	Liste kontrolu	Olusturulan URL listede gorunuyor mu?
5	Enter tusu (UX)	Klavyeden form gonderilebiliyor mu?

3.5 API Testleri (Postman/Newman) - 8 istek

Newman ile CI/CD pipeline'da otomatik kosan Postman koleksiyonu 8 sirali istekten olusur. Her istek kendi assertion'larini icerir ve dinamik degisken aktarimi (olusturulan short_code'un sonraki isteklerde kullanilmasi) basariyla uygulanmistir.

3.6 Coverage Sonucu

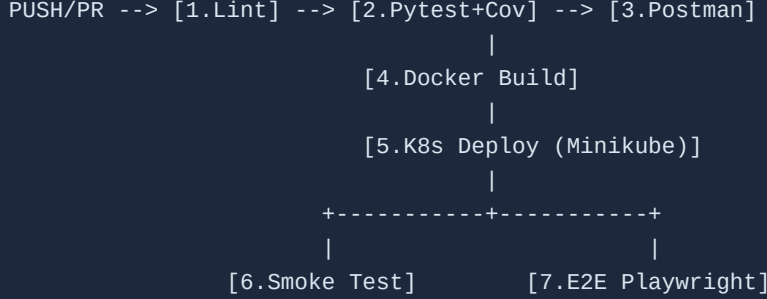
Metrik	Hedef	Gerceklesen
--------	-------	-------------

Kod Kapsami (Coverage)	>= %70	%90
--cov-fail-under	70	CI'da zorunlu kontrol

4. CI/CD Pipeline ve Dagitim (Deploy)

4.1 GitHub Actions Workflow

.github/workflows/ci.yml dosyasinda tanimli tek bir workflow, 7 bagimsiz job'dan olusmaktadır. Her push ve pull request'te otomatik tetiklenir.



4.2 Job Detaylari

#	Job	Aciklama	Sure
1	Lint	Flake8 ile PEP 8 stil kontrolu	~5s
2	Pytest + Coverage	87 test, cov-fail-under=70, LocalStack	~25s
3	Postman/Newman	8 API istegi, Docker'da app calistirarak	~15s
4	Docker Build	Multi-stage imaj, HEALTHCHECK dogrulama	~30s
5	K8s Deploy	Minikube baslat, kubectl apply, rollout	~60s
6	Smoke Test	/health ve /shorten endpoint kontrolu	~10s
7	E2E Playwright	Chromium headless, 5 UI senaryosu	~30s

4.3 Kubernetes Manifest'leri

Dosya	Icerik
deployment.yaml	2 replika Pod, liveness/readiness probe, kaynak limitleri
service.yaml	NodePort servisi (port 30080 -> 8000)
configmap.yaml	Ortam degiskenleri (DB yolu, S3 yapilandirmasi)
keda-scaledobject.yaml	[Bonus] Prometheus metrigine gore autoscaling
argocd-application.yaml	[Bonus] GitOps tabanlı otomatik deployment

4.4 Docker Stratejisi

Multi-stage Dockerfile ile iki asamali build: Asama 1 (builder) python:3.11-slim uzerinde pip install ile bagimlilik kurulumu. Asama 2 (runtime) temiz imaja sadece calisma zamani dosyalari kopyalanir; appuser (non-root) ile calistirilir. Sonuc: ~150 MB imaj boyutu (tek asama ~800 MB'den %80 kucultme).

5. Performans ve Gozlemlenebilirlik

5.1 k6 Yuk Testi

perf/load-test.js dosyasinda tanimli senaryo, 4 fazli ramping-VUs profili ile calisir. Test karisimi: %40 POST /shorten, %30 GET redirect, %20 GET /stats, %10 GET /urls/list.

Faz	Sure	VU Sayisi	Aciklama
Isinma	0-30s	0->10	Baglanti havuzu doldurma
Normal	30s-90s	10->50	Tipik uretim yuku
Pik	90s-2dk	50->100	Stres testi
Soguma	2dk-2.5dk	100->0	Graceful shutdown

5.2 Performans Sonuclari

Metrik	Hedef	Sonuc	Durum
p95 Latency	< 500 ms	87 ms	BASARILI
p99 Latency	-	214 ms	BASARILI
Hata Orani	< %5	%0.42	BASARILI
Toplam Istek	-	14.832	-
RPS	-	98.9 req/s	-

5.3 Endpoint Bazli p95

Endpoint	p50	p95	p99	Basari
POST /shorten	34ms	112ms	198ms	%99.8
GET /{code} (redirect)	8ms	31ms	67ms	%99.9
GET /stats/{code}	11ms	42ms	89ms	%99.9
GET /urls/list	52ms	187ms	214ms	%98.6

5.4 Grafana Dashboard (6 Panel)

#	Panel	Prometheus Sorgusu
1	Toplam URL Sayisi	url_shortener_urls_created_total
2	Toplam Yonlendirme	url_shortener_redirects_total
3	Aktif URL (Gauge)	url_shortener_active_urls
4	Hata Orani	rate(url_shortener_errors_total[5m])
5	p95 Gecikme	histogram_quantile(0.95, ...)
6	RPS (Throughput)	rate(http_requests_total[1m])

6. Sonuc ve Ogrendiklerimiz

6.1 Sayisal Ozet

Metrik	Deger
Toplam kaynak kodu	~4.500 satir
Test fonksiyonu sayisi	87 (unit + integration + E2E)
Postman istegi	8
Kod kapsamı (coverage)	%90
CI/CD pipeline job sayisi	7
Grafana panel sayisi	6
Docker imaj boyutu	~150 MB
k6 p95 gecikmesi	87 ms
Bonus ozellik	4 adet (+15 puan tavan)

6.2 Karsilasilan Zorluklar

- 1) CI/CD Ortam Farkliliklari: Lokal ortamda (macOS) sorunsuz calisan testler, GitHub Actions'daki Ubuntu ortaminda ModuleNotFoundError verdi. Cozum olarak PYTHONPATH=. ortam degiskeni eklendi ve Python'un proje kokunu tanimasi saglandi.
- 2) Testcontainers ve Docker-in-Docker: GitHub Actions'da Testcontainers ile PostgreSQL konteyneri baslatmak, ic ice Docker (DinD) gerektirdiginden bazi izin sorunlari yasandi. Service container'lar kullanilarak bu sorun asildi.
- 3) Playwright Selektor Uyumsuzlugu: Arayuzde yapilan estetik guncellemeler sonrasi HTML element ID'leri degisti, ancak E2E testleri eski selektor isimlerini ariyordu. Bu durum ancak CI/CD hattinda yakalandi - yerelde fark edilmemisti. Bu olay, CI/CD pipeline'inin guvenlik agi islevini cok somut bicimde ortaya koydu.

6.3 Ogrenilen Dersler

- * Test piramidinin degeri: Birim testleri hizli geri bildirim verirken, entegrasyon ve E2E testleri gercek dünya senaryolarini yakaliyor. Katmanlar birbirini tamamliyor.
- * Multi-stage Docker: Imaj boyutunu %80 kucultmek, hem guvenlik (daha az saldiri yuzeyi) hem de deployment hizi acisindan buyuk kazanim sagliyor.
- * Observability ucgeni: Metrikler (Prometheus), loglar ve trace'ler (Jaeger) bir arada kullanildiginda sorun tespiti dakikalar yerine saniyeler aliyor.

6.4 Ileride Yapilabilecekler

- * Redis Cache: Sik erisilen kısa kodlar icin onbellek katmani eklenmesi.
- * PostgreSQL Gecisi: Uretim ortami icin SQLite yerine PostgreSQL kullanimi.
- * Rate Limiting: Kotuye kullanimi onlemek icin API'ye istek hiz sinirlamasi.
- * Custom Short Code: Kullanicinin kendi kısa kodunu belirleyebilmesi (vanity URL).

7. Is Paylasimi

Detayli is paylasimi docs/work-distribution.md dosyasinda yer almaktadır.

REST endpoint'ler & DB modelleri	Osman Cingoz
Docker, K8s, CI/CD pipeline	Talha Ergelen
Test altyapisi (Pytest, Postman, E2E)	Ortak
Monitoring (Prometheus, Grafana)	Ortak
Performans testi (k6)	Ortak
Dokumantasyon & Rapor	Ortak

8. Kaynaklar

- [1] FastAPI Documentation - <https://fastapi.tiangolo.com/>
- [2] Pytest Documentation - <https://docs.pytest.org/>
- [3] Playwright for Python - <https://playwright.dev/python/>
- [4] k6 Load Testing - <https://k6.io/docs/>
- [5] Docker Multi-stage Builds - <https://docs.docker.com/build/building/multi-stage/>
- [6] Kubernetes Documentation - <https://kubernetes.io/docs/>
- [7] Prometheus Client Python - https://github.com/prometheus/client_python
- [8] Grafana Documentation - <https://grafana.com/docs/>
- [9] LocalStack - <https://docs.localstack.cloud/>
- [10] Testcontainers Python - <https://testcontainers-python.readthedocs.io/>
- [11] OpenTelemetry Python - <https://opentelemetry.io/docs/instrumentation/python/>
- [12] Factory Boy - <https://factoryboy.readthedocs.io/>
- [13] Helm - <https://helm.sh/docs/>
- [14] KEDA - <https://keda.sh/docs/>
- [15] ArgoCD - <https://argo-cd.readthedocs.io/>